BMSLO 701-1338-00L 2026

Box models, transport processes and more – a Systems Analysis reminder

Meike Vogt, meike.vogt@env.ethz.ch
Martin Schmid, martin.schmid@eawag.ch

Download:

Tasks and Templates:

- 1box_model_template.py
- 2box_model.py
- simple_pde.py
- pde.py

Solutions:

- 1box_model.py
- simple_pde_corr.py
- simple_fipy_corr.py

Download:

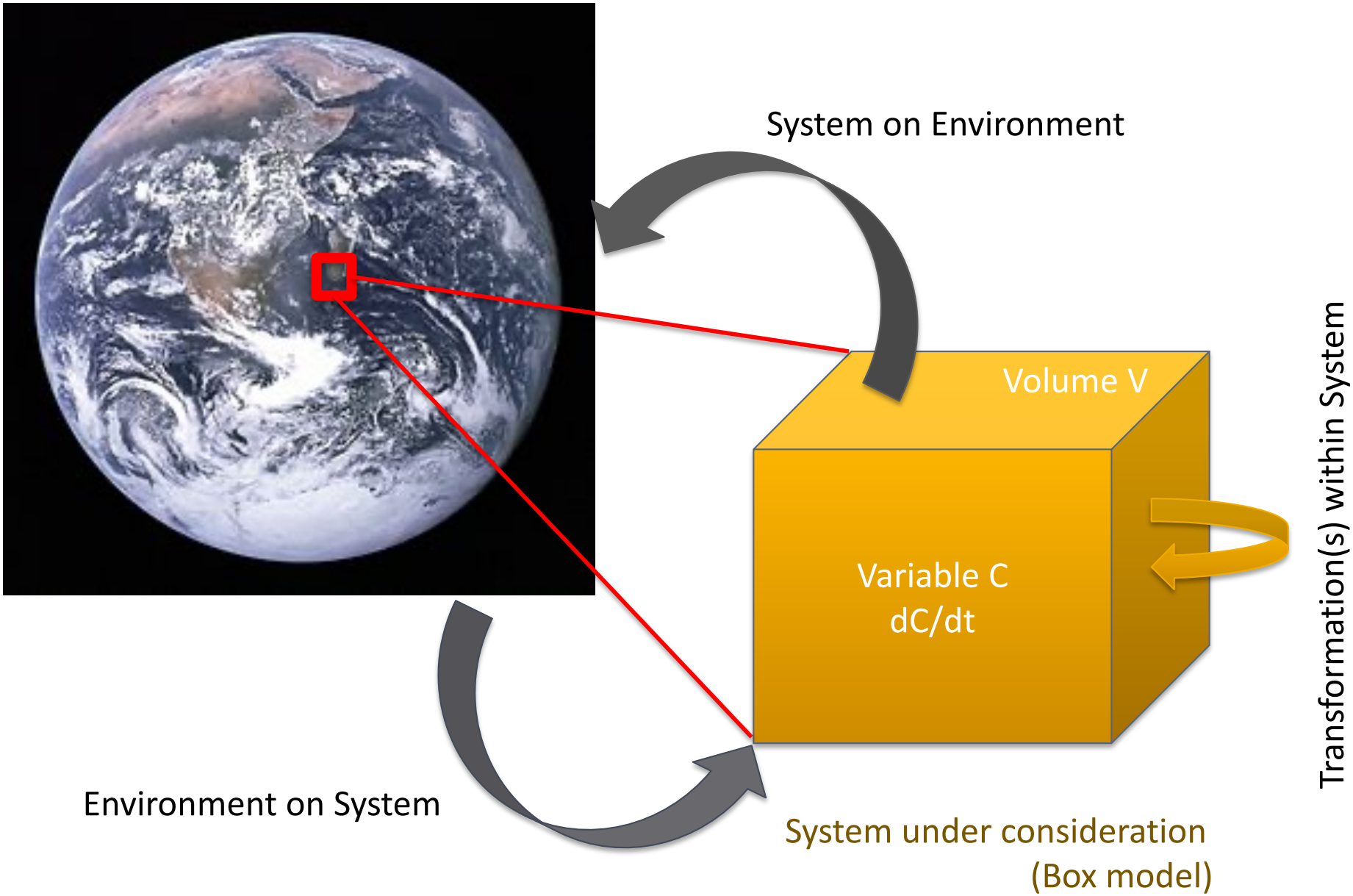
Tasks and Templates (Jupyter notebooks):

- 1box_model_template.ipynb
- 2box_model.ipynb
- simple_pde.ipynb
- pde.py

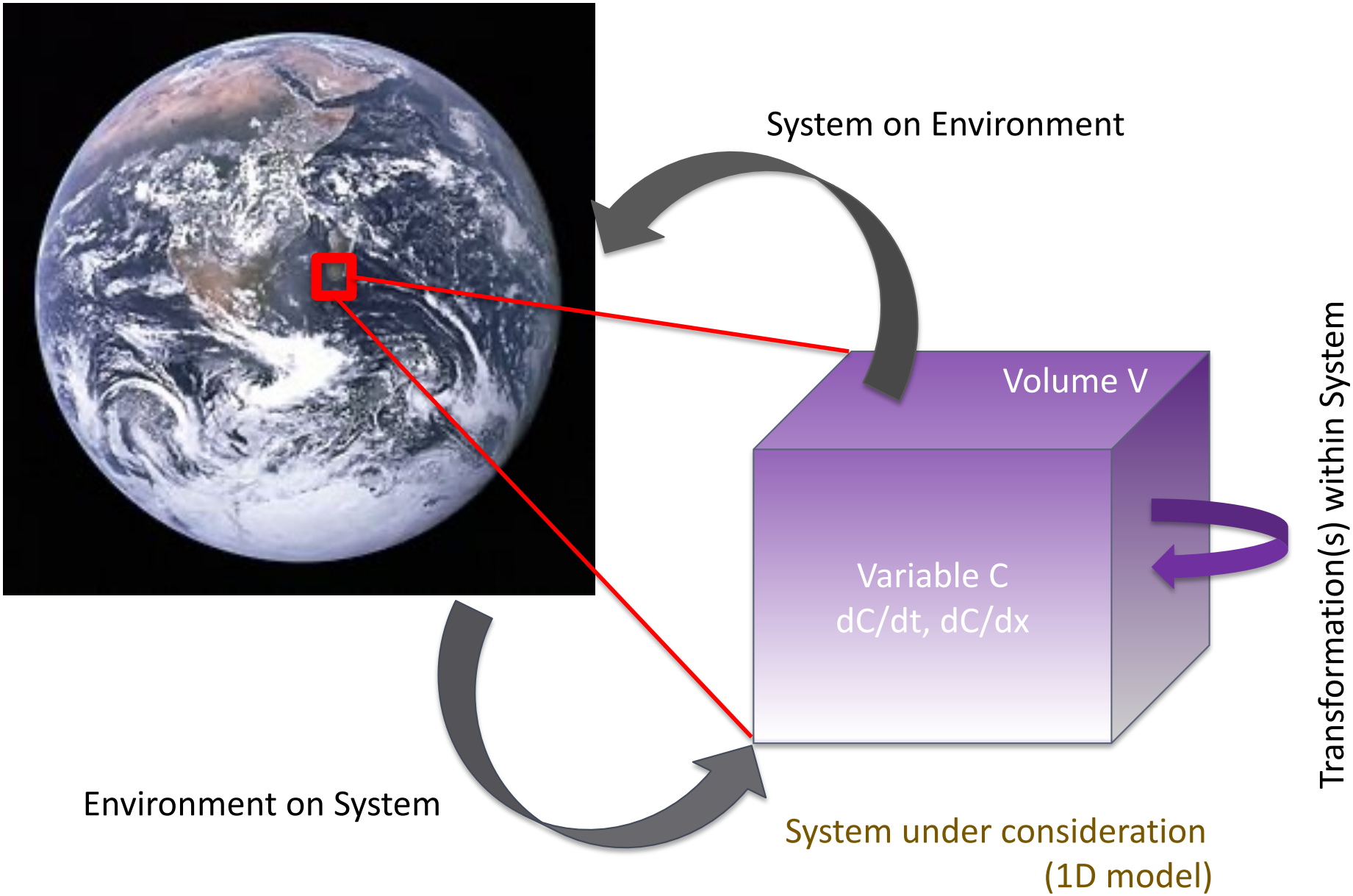
Solutions:

- 1box_model.ipynb
- simple_pde_corr.ipynb
- simple_fipy_corr.ipynb

Reminder: Mathematics III – Systems Analysis



Reminder: Mathematics III – Systems Analysis



Reminder: Mathematics III – Systems Analysis

We will be using Python libraries to solve systems of equations of the form:

1. $f(x, y(x), dy(x)/dx, d^2y(x)/dx^2, \ldots, d^ny(x)/dx^n) = 0$;

(**ordinary differential equations; ODE**)

e.g. $\frac{dC}{dt} = aC(t) - b(t)$ (first order linear ordinary differential equation)

"differential equations containing functions of a single variable and their derivatives"

2. $f(x_1,..x_n; u(x), \partial u(x)/\partial x_1,..\partial u(x)/\partial x_n; \partial^2 u(x)/\partial x_1 \partial x_1, \ldots, \partial^2 u(x)/\partial x_1 \partial x_n, \ldots) = 0$;

(**partial differential equations; PDE**)

e.g. $\frac{dC}{dt} = \frac{d^2C}{dx^2}$ (diffusion equation)

"differential equations containing multivariable functions and their partial derivatives"

Solvers for ordinary differential equations (ODEs) in Python

One specific method from scipy: solve_ivp()

***scipy.integrate.solve_ivp**(fun, t_span, y0, method='RK45', t_eval=None)*

Solve an initial value problem for a system of ODEs.

This function numerically integrates a system of ordinary differential equations given an initial value:

```
dy / dt = f(t, y)
y(t0) = y0
```

Here t is a one-dimensional independent variable (time), y(t) is an n-dimensional vector-valued function (state), and an n-dimensional vector-valued function f(t, y) determines the differential equations. The goal is to find y(t) approximately satisfying the differential equations, given an initial value y(t0)=y0.

Specifies time points to evaluate at.
solve_ivp chooses points if not specified.

```python
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt

def exponential_decay(t,y): return -0.5 * y

solution = solve_ivp(exponential_decay, [0,10],[2], method='RK45')

plt.plot(solution.t, solution.y[0], 'r+')
```

Written entirely in Python and simple to use, but slow.

https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html

Solvers for partial differential equations (PDEs) in Python

One specific method written for our course: pde_solver()

sol=pde_solver(pdefun,bc_left, bc_right,icfun,xmesh,tspan)

Symmetry | Function | Boundary conditions | Initial conditions | Space discretization | Time discretization

pde_solver solves certain parabolic and elliptic PDEs in one spatial dimension x and time dimension t.
Drop pde.py into your working folder and import it!

Solves equations of the form:

$$c\left(x,t,u,\frac{\partial u}{\partial x}\right)\frac{\partial u}{\partial t} = \frac{\partial}{\partial x}\left(f\left(x,u,t,\frac{\partial u}{\partial x}\right)\right) + s\left(x,u,t,\frac{\partial u}{\partial x}\right)$$

With boundary conditions of the form:

Dirichlet: $u(t) = p(x) + q(t, u(t), x) * t$

Neumann: $\frac{\partial u(t)}{\partial x} = f(t, u(t), x)$

```
# ---------------------Execution---------------------------
# Call pde_solver. It needs the following arguments:
# simple_pde: function containing the PDEs
# BC_left: BoundaryCondition containing conditions for z = 0
# BC_right: BoundaryCondition containing conditions for z = Zmax
# simple_IC: function containing initial condition for t=0 at all z
# z: z grid
# t: t grid
sol = pde_solver(simple_pde, BC_left, BC_right, simple_IC, z, t)
# Output: 3D array. [PDE, Z, T]
```

Dirichlet conditions: Give a variable's value at boundaries (e.g. concentrations)
Neumann conditions: Give a variable's change at boundaries (e.g. fluxes)

How to define your model equations

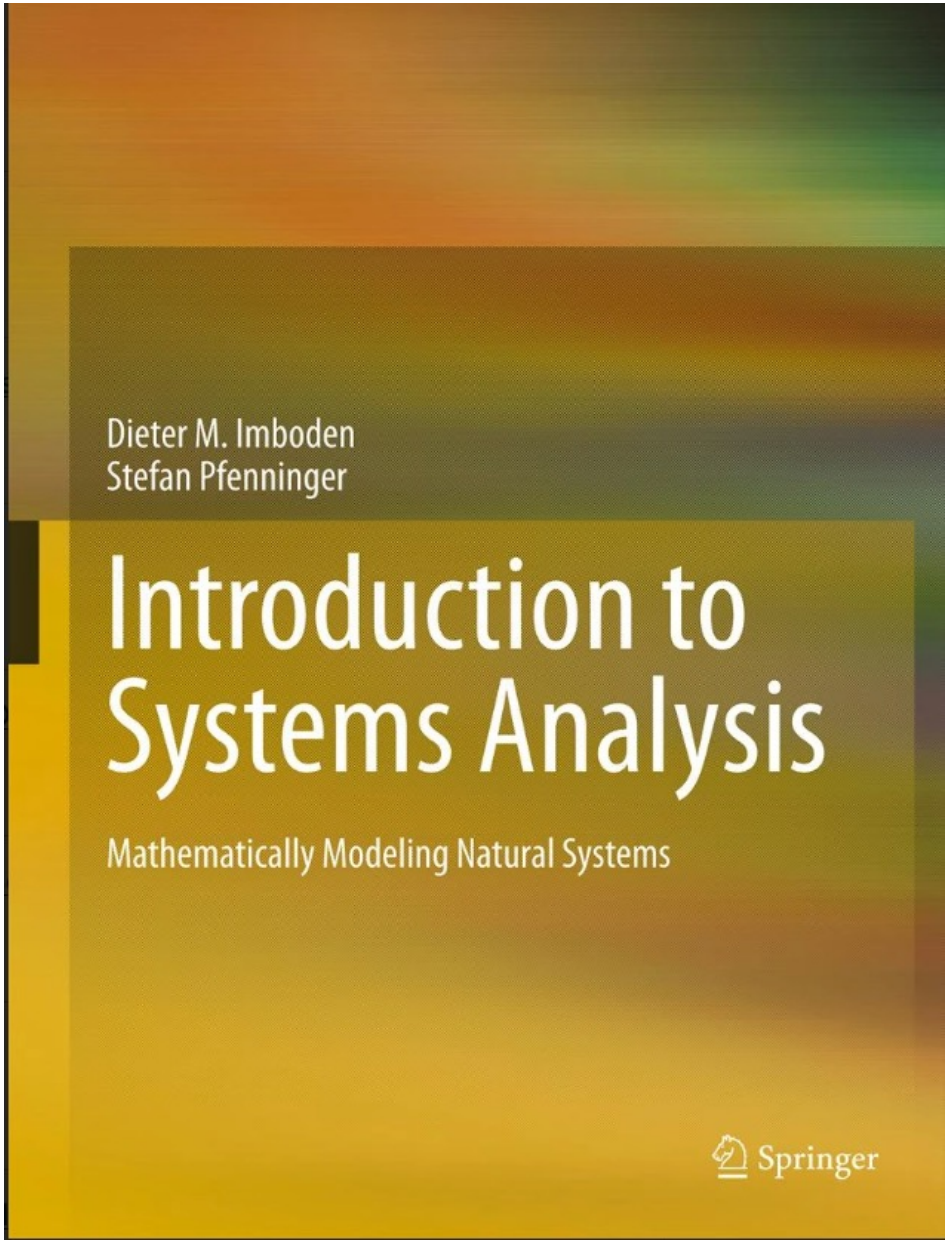
Remember the D-USYS courses:

I. Mathematik III: Systemanalyse
(Knutti/Wernli)

German version:
Systemanalyse
(D. Imboden & S. Koch)
https://link.springer.com/book/10.1007/978-3-642-55667-8

English version:
Introduction to Systems Analysis
(D. Imboden & S. Pfenninger)
https://link.springer.com/book/10.1007/978-3-642-30639-6

II. Modellierung aquatischer Ökosysteme
(Reichert)



How to set up a(ny) (box) model

1. Formulate questions to be solved
2. Define boundaries of your system
3. Define grid / resolution
4. Set up the mass balance equation(s)
5. Set up the numerical model
6. Solve and test the model
7. Process and analyze the results
8. Interpret findings

How to set up a (box) model

1. Formulate questions to be solved

„If we burn fossil fuels and add CO_2 to the atmosphere, how much of it is going to remain there“?

2. Define boundaries of your system

The sky is the limit on Planet Earth. Volume of atmosphere, ocean, terrestrial biosphere.

3. Define grid / resolution

Grid: E.g. 3-box model with atmospheric, oceanic and terrestrial CO_2 reservoir; temporal resolution: IPCC time period 2000 - 2100

4. Set up the mass balance equation(s)

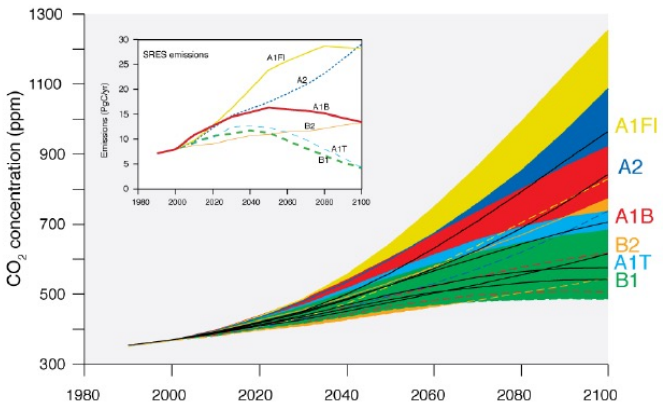
$d\,CO_2 / dt = \Phi_{foss} - F_{as} - F_{ab}$

5. Set up the numerical model

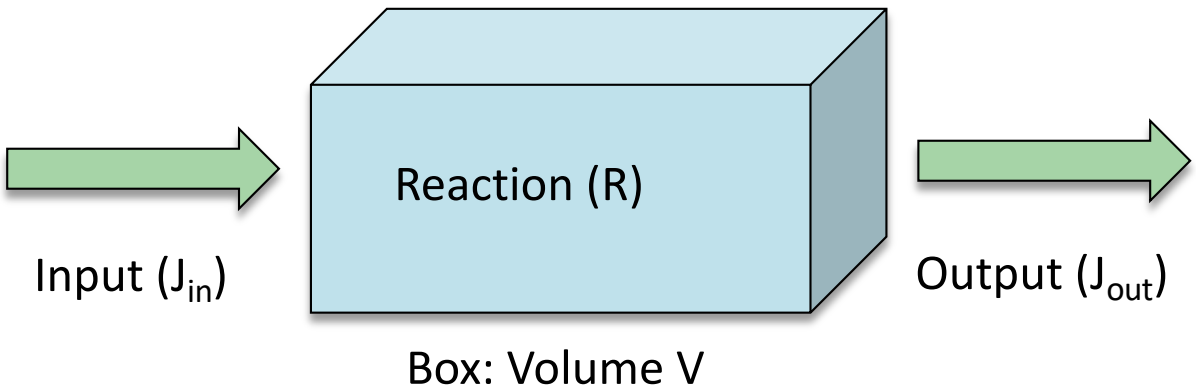
6. Solve and test the model

$$CO_2(t) = \int \Phi_{foss}\, dt - \int F_{as}\, dt - \int F_{ab}\, dt$$

7. Process and analyze the results



Box models: 1-box model (mixed reactor) with one state variable



Mass balance equation:

$$\frac{dM}{dt} = J_{in}(t) - J_{out} + \mathrm{R}$$

e.g. [kg/t]

$$V\frac{dC}{dt} = Q\,C_{in}(t) - QC + k*CV$$

E.g., Systems Analysis:
Linear Reactor
R = k * CV

e.g. [m^3/t]

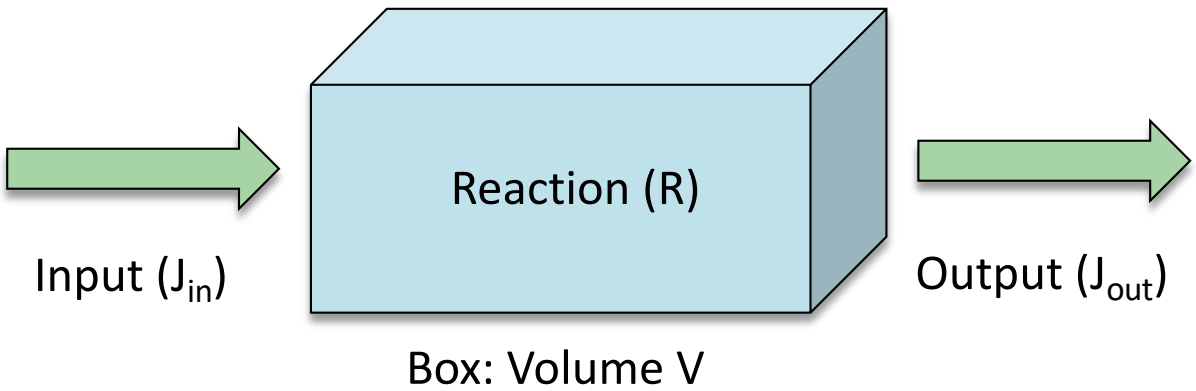
$$\frac{dC}{dt} = \frac{Q}{V}C_{in}(t) - \frac{Q}{V}C + k*C$$

$$\frac{dC}{dt} = k_w C_{in}(t) - (k_w + k_r)*C$$

1st order inhomogeneous linear ODE
k_w: rate of exchange (T^{-1})
k_r: -k; reaction rate (T^{-1})
$k_{tot} = (k_w + k_r)$: total rate of change

E.g., Systems Analysis:
C_{in} [t] = j_{in} sin(ωt)
C_{in} [t] = $e^{\beta t}$

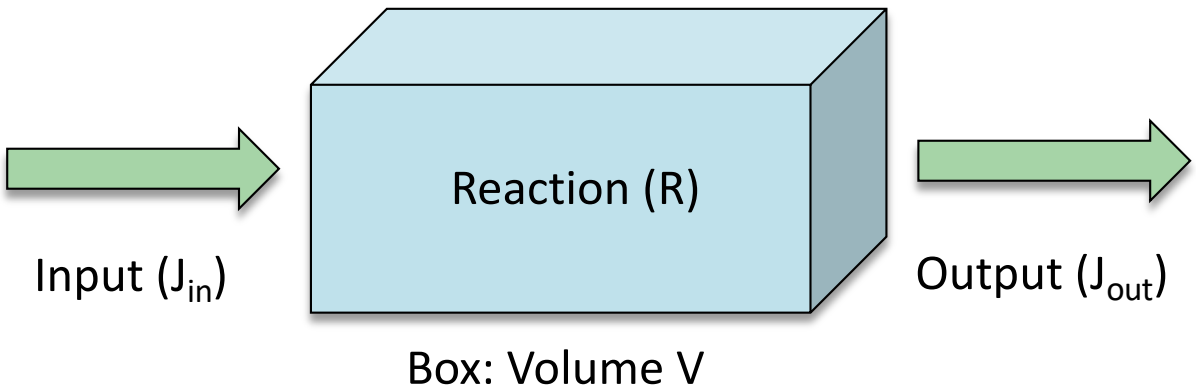
Box models: 1-box model (mixed reactor) with one state variable



Example
Phosphorus balance of a lake with constant Volume V
Input = Inflow concentration (P_{in}) x Inflow (Q_{in} = Q)
Output = Concentration in lake (P) x Outflow (Q_{out} = Q)
Net sedimentation is proportional to concentration in lake (Sed = k_{Sed} x P)

Exercise 1: Determine the model equations for this system

Box models: 1-box model (mixed reactor) with one state variable



Example
Phosphorus balance of a lake with constant Volume V
Input = Inflow concentration (P_{in}) x Inflow (Q_{in} = Q)
Output = Concentration in lake (P) x Outflow (Q_{out} = Q)
Net sedimentation is proportional to concentration in lake (Sed = k_{Sed} x P)

$$V\frac{dP}{dt} = QP_{in} - QP - Vk_{sed}P$$

$$\frac{dP}{dt} = k_w P_{in} - k_w P - k_{sed}P \Leftrightarrow k_w = \frac{Q}{V}$$

$$\frac{dP}{dt} = k_w P_{in} - (k_w + k_{sed})P$$

The different types of variables

Independent variables

Usually time and/or space

$$\frac{dP}{dt} = k_w P_{in} - k_w P - k_{sed} P$$

State variables

The state variables completely describe the current state of the system.
For each state variable, you need to define one differential equation that describes the change of its value as a function of the independent variables.

$$\frac{dP}{dt} = k_w P_{in} - k_w P - k_{sed} P$$

Parameters

Parameters are used to define the properties of the model.

$$\frac{dP}{dt} = k_w P_{in} - k_w P - k_{sed} P$$

Take care of the units!!

Python can't check your units, so you have to check them yourself!

Safest approach: use only consistent units

1) Define basic (SI) units

Time	**Space**	**Mass**
s	m	kg

2) Use consistent derived units

Flow	**Concentration**
$m^3\ s^{-1}$	$kg\ m^{-3}$

etc…

Disadvantage: some properties might have unfamiliar units or unpractical values.

Example

Time	**Space**
d	m

⇨ standard gravity $g = 7.32*10^{10}\ m\ d^{-2}$

($g = 9.81\ m/s^2$)

Alternative approach: use preferred/standard units
Disadvantage: you have to take care of unit conversion, significant risk for errors

How to define model parameters?

You know the value of the parameter, and the uncertainty is negligible
Example: lake volume
⇨ Set the parameter to the known value and forget about it ☺

You have some idea of the value, but there is a significant uncertainty.
Examples: observed concentrations in inflows, reaction rates from laboratory experiments
⇨ Set the value to your best estimate based on the available knowledge.
⇨ Test the sensitivity of the model results to the value of the parameter (*lecture on sensitivity analysis, 17.04.2026*).
⇨ Reduce the uncertainty of the parameter value based on observations of the state of the system (*lecture on parameter estimation, 17.04.; lecture on model evaluation 24.04.*).

You don't know the value of the parameter at all.
⇨ Try to measure the value of the parameter.
⇨ Try to estimate the parameter based on observations of the state of the system (*lecture on parameter estimation, 17.04.2026*).

Note: parameters can also be variable in time and/or space!

Basic functions to include in your models (BMSLO_SA_Reminder*)

Exponential decay

k: decay rate [T^{-1}]

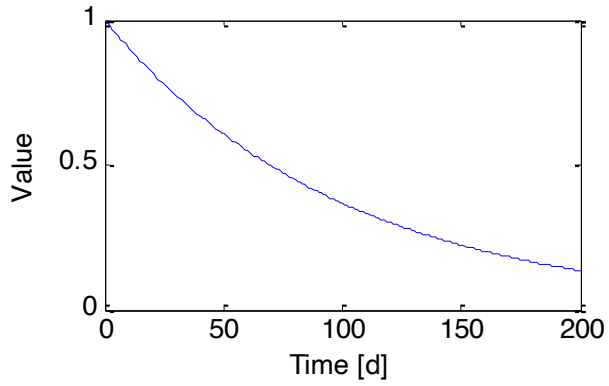
Examples:

radioactive decay

first-order reaction

$$\frac{dC}{dt} = -kC$$

```
def exp_decay(t,C):
    k = 0.01 # decay rate
    return -k*C
```



Exponential growth

k: growth rate [T^{-1}]

Examples:

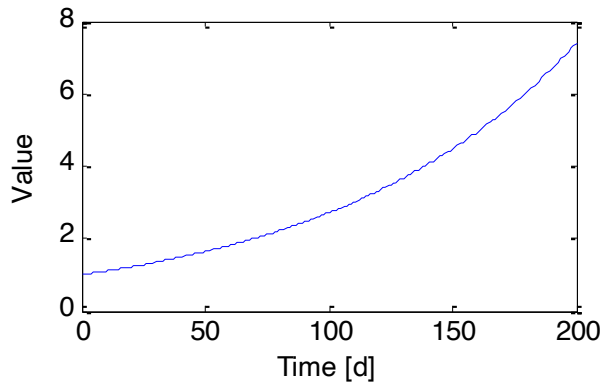
to be used carefully

most growth is limited

(see next examples).

$$\frac{dC}{dt} = kC$$

```
def exp_growth(t,C):
    k = 0.01 # growth rate
    return k*C
```



Logistic growth

r: maximum growth rate [T^{-1}]

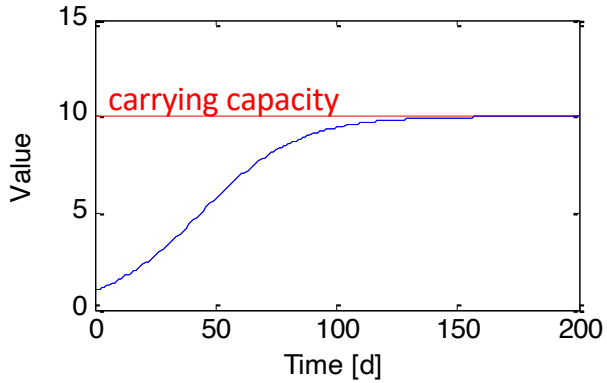
K: carrying capacity

(same unit as C)

Examples: population growth

$$\frac{dC}{dt} = rC\frac{(K-C)}{K}$$

```
def log_growth(t,C):
    r = 0.05 # maximum growth rate
    K = 10 # carrying capacity
    return r*C*(K-C)/K
```



Basic functions to include in your models (BMSLO_SA_Reminder*)

Michaelis-Menten / Monod

r: maximum growth rate [T^{-1}]

K: half-saturation constant

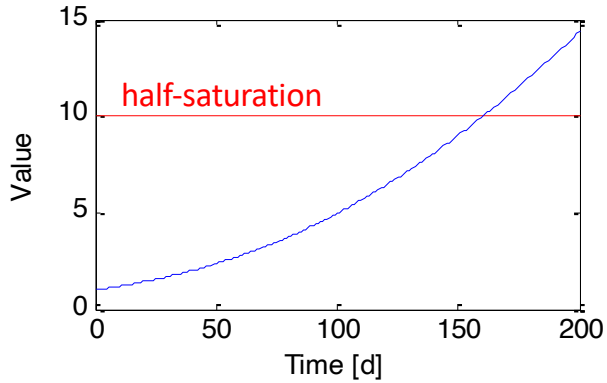
Examples:

growth/reaction

limited by substance conc.

$$\frac{dC}{dt} = r\frac{C}{C+K}$$

```
def monod(t,C):
    r = 0.2 # maximum growth rate
    K = 10 # half-saturation
    return r*C/(K+C)
```



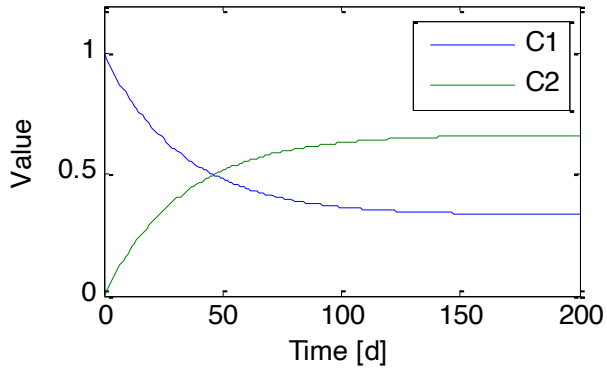
Chemical equilibrium

k_1 and k_2: reaction rates [T^{-1}]

$$\frac{dC_1}{dt} = -k_1C_1 + k_2C_2$$

$$\frac{dC_2}{dt} = -k_2C_2 + k_1C_1$$

```
def equilib(t,C):
    k1 = 0.02 # kinetic constant 1
    k2 = 0.01 # kinetic constant 2
    dCdt1 = -k1*C[0]+k2*C[1]
    dCdt2 = -k2*C[1]+k1*C[0]
    return [dCdt1,dCdt2]
```



How to define and solve a 1-box model in Python (1box_model.py)

$$\frac{dP}{dt} = \frac{Q}{V} * P_{in} - \frac{Q}{V} P - k_{sed} P$$

Don't forget your imports!

```
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt
```

1) Define differential equation as a function

```
# Define lake function
def lake_1box(t, P):
    return Q / V * Pin - Q / V * P - k * P
```

Function name
Independent variable
State variable(s)
Model Parameters

2) Define time grid

```
# Define time grid
tini = 0  # initial time [d]
t_nodes = 201  # number of time nodes [d]
tend = 200  # end time [d]
t_space = np.linspace(tini, tend, t_nodes)
```

3) Define model parameters

```
# Define model parameters
V = 100000  # Lake Volume [m3]
Q = 5000  # Inflow/Outflow [m3/d]
Pin = 100e-6  # Phosphorus concentration in the inflow [kg m-3]
k = 0.01  # Sedimentation rate [d-1]
```

4) Define initial condition(s)

```
# Define initial condition
Pini = 10e-6  # initial P concentration in the lake [kg m-3]
```

5) Calculate solution

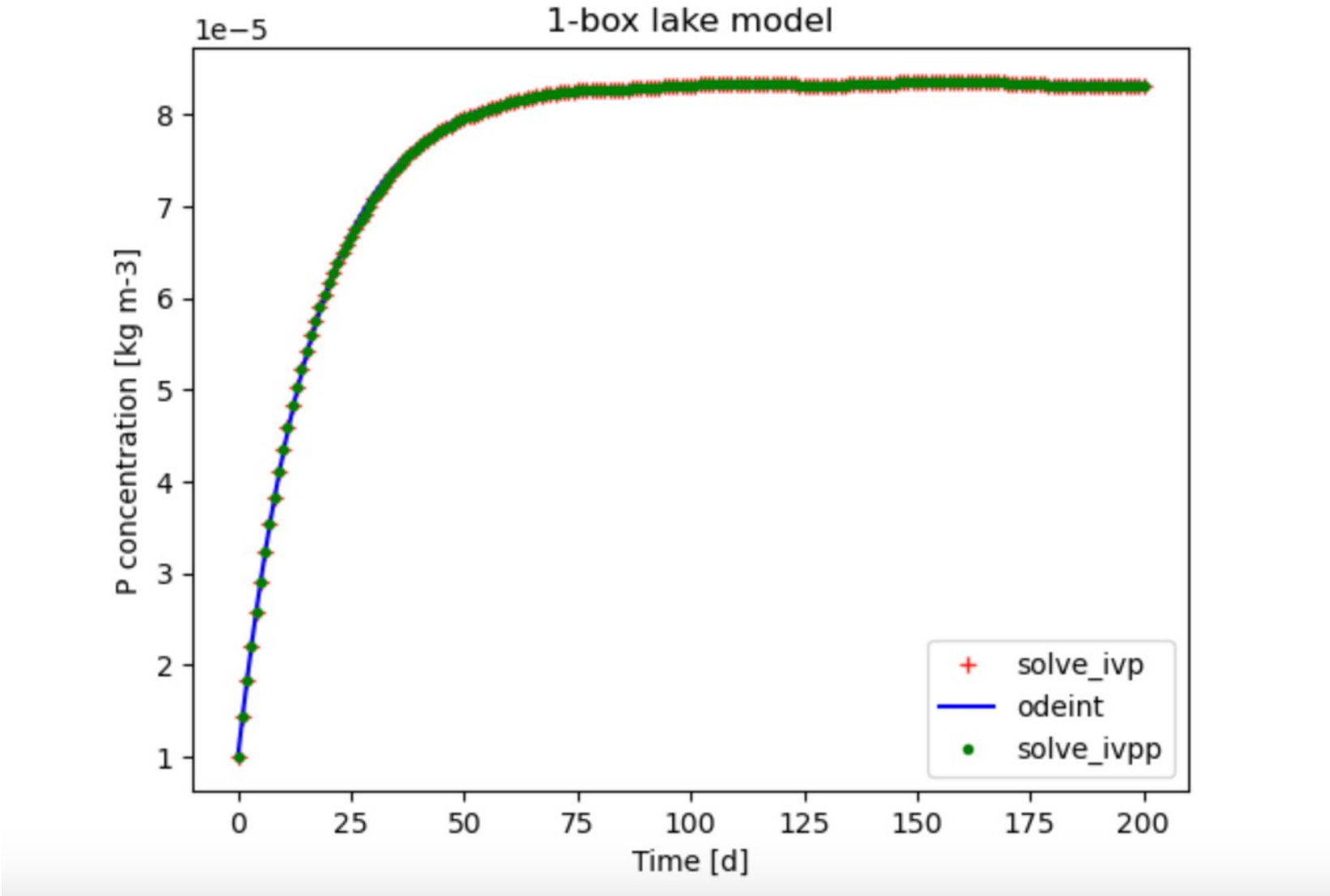
```
# calculate solution
# remember to switch the method to one
# which can solve stiff problems if your problem is stiff!
solved_ivp = solve_ivp(lake_1box, [tini, tend],
                       [Pini], method='RK45', t_eval=t_space)
```

6) Plot or export results

```
# plot results
plt.figure()
plt.plot(solved_ivp.t, solved_ivp.y[0], 'r')
plt.show()
```

How to define and solve a 1-box model in Python (1box_model.py)

$$\frac{dP}{dt} = \frac{Q}{V} * P_{in} - \frac{Q}{V} P - k_{sed} P$$

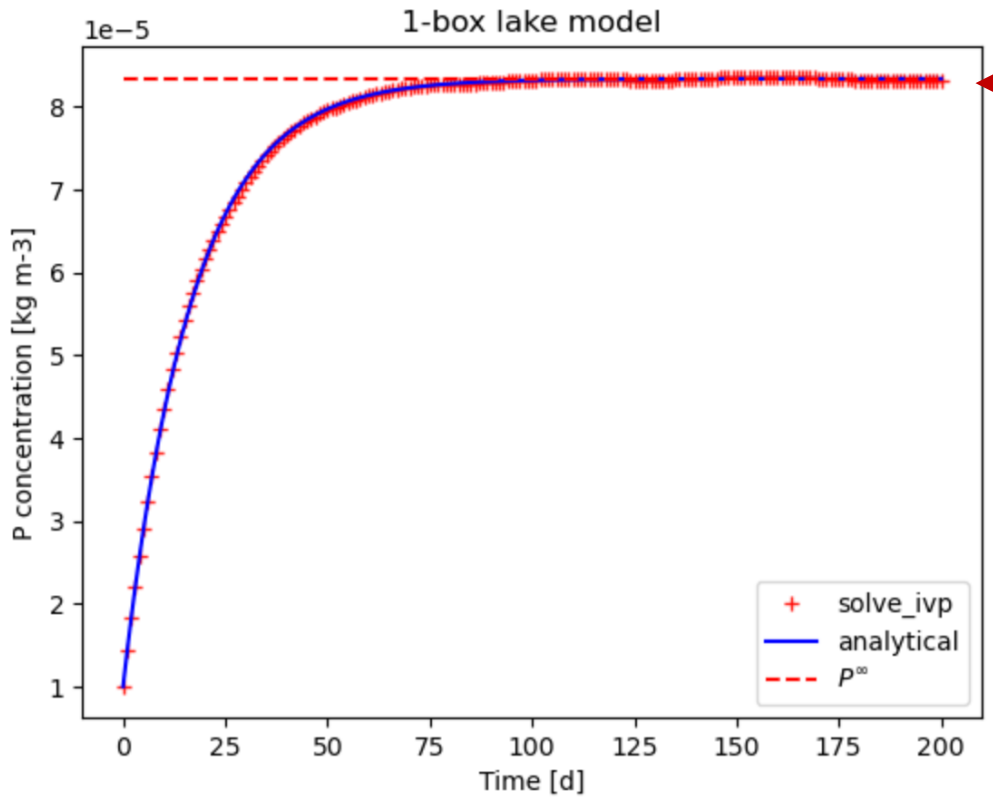


How to check the solution of a 1-box model in Python (1box_model.py)

Step 8: Use the analytical solution to check your solution.

This is $P(t) = P_{in} * \exp^{-kt} + P^{\infty} * (1 - \exp^{-kt})$

```
def mylakeequation(t, Pini, Pinf, kw, k):
    return Pini * np.exp(-(kw+k) * t) + Pinf * (1 - np.exp(-(kw+k) * t))

mylakesolution = mylakeequation(t_space, Pini, Pinf, kw, k)
```



$P^{\infty} = \frac{k_w}{k_w+k} * P_{in}$

Exercise 2: Define & solve a 1-box model (1box_model_template.py)

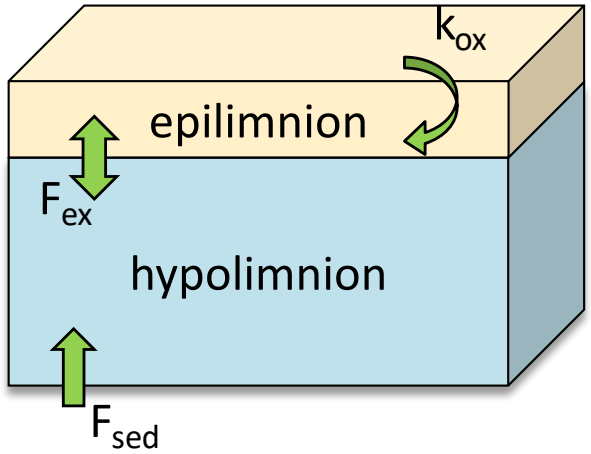
$$\frac{dP}{dt} = \frac{Q}{V} * P_{in} - \frac{Q}{V} P - k_{sed} P$$

```python
import numpy as np
import matplotlib.pyplot as plt
# 2 different ode solvers which we can compare
from scipy.integrate import solve_ivp
from scipy.integrate import odeint

# Define time grid
tini = 0  # initial time [d]
dt = 1  # output time step [d]
tend = 200  # end time [d]
t_space = np.linspace(tini, tend, int((tend - tini) / dt + 1))

# Define model parameters
V = 100000  # Lake Volume [m3]
Q = 5000  # Inflow/Outflow [m3/d]
Pin = 100e-6  # Phosphorus concentration in the inflow [kg m-3]
k = 0.01  # Sedimentation rate [d-1]

# Define initial condition
Pini = 10e-6  # initial P concentration in the lake [kg m-o]
```

Box models: 2-box model with one state variable (2box_model.py)

Example
Methane in a lake
2 boxes: epilimnion (oxic), hypolimnion (anoxic)

Methane is released from the sediment (F_{sed})
and oxidized in the epilimnion (first-order, rate k_{Ox})



Exchange (F_{ex}) between hypolimnion and epilimnion:
Assume concentration gradient over 5 m (Δz), and vertical diffusivity K_z of 10^{-6} m^2 s^{-1}

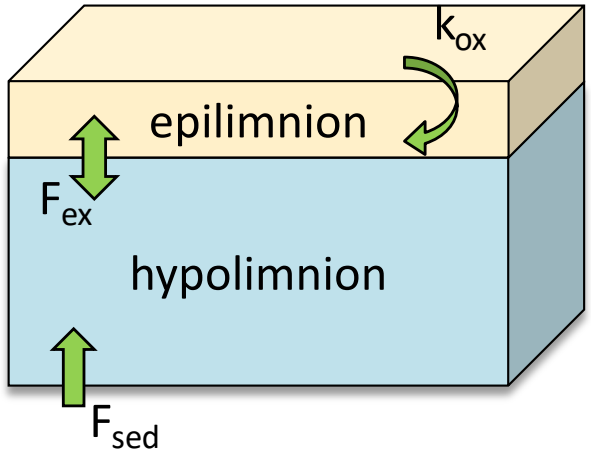
$$F_{ex} = K_z \frac{C_h - C_e}{\Delta z}$$

Exercise 3: set up the mass balance equations for the epilimnion and the hypolimnion

Box models: 2-box model with one state variable (2box_model.py)

Example
Methane in a lake
2 boxes: epilimnion (oxic), hypolimnion (anoxic)

Methane is released from the sediment (F_{sed})
and oxidized in the epilimnion (first-order, rate k_{Ox})



Exchange (F_{ex}) between hypolimnion and epilimnion:
Assume concentration gradient over 5 m (Δz), and vertical diffusivity K_z of 10^{-6} m^2 s^{-1}

$$F_{ex} = K_z \frac{C_h - C_e}{\Delta z}$$

Exercise 3: set up the mass balance equations for the epilimnion and the hypolimnion

$$V_h \frac{dC_h}{dt} = AF_{sed} - AK_z \frac{C_h - C_e}{\Delta z}$$

$$V_e \frac{dC_e}{dt} = AK_z \frac{C_h - C_e}{\Delta z} - V_e k_{ox} C_e$$

How to define and solve the 2-box model in Python (2box_model.py)

1) Import libraries

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
```

2) Define time grid

```
# Define time grid
tini = 0  # initial time [d]
t_nodes = 201  # time node number [d]
tend = 200  # end time [d]
t_space = np.linspace(tini, tend, t_nodes)
```

More boxes and/or state variables can be added analogously!

3) Define model parameters

```
# Define model parameters
Vepi = 10000  # Epilimnion Volume [m3]
Vhypo = 90000  # Hypolimnion Volume [m3]
A = 10000  # Lake area (for simplicity assumed constant) [m2]
Fsed = 0.02  # Flux of methane from sediment [mol m-2 d-1]
kOx = 0.05  # Oxidation rate [d-1]
Kz = 86400 * 1e-5  # Vertical diffusivity [m2 d-1]
dz = 5  # thickness of metalimnion [m]
```

4) Define initial condition(s)

```
# Define initial condition
Cini = [0, 0.01]  # initial CH4 conc. in the epi- and hypolimnion [mol m-3]
```

5) Define differential equations as a function

```
# Define function
def lake_2box(t, C):
    depidt = A / Vepi * Kz * (C[1] - C[0]) / dz - kOx * C[0]
    dhypodt = A / Vhypo * Fsed - A / Vhypo * Kz * (C[1] - C[0]) / dz
    return [depidt, dhypodt]
```

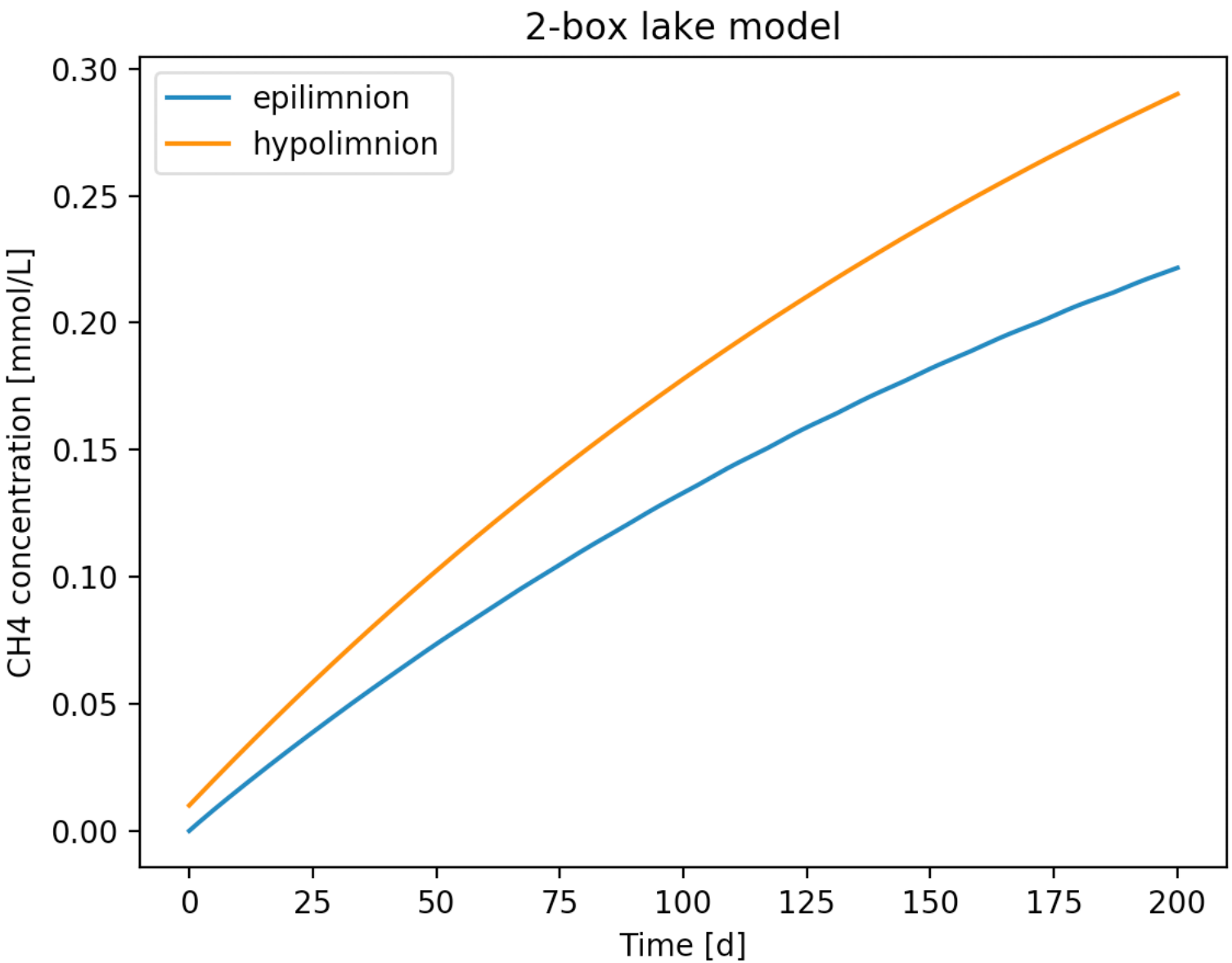
6) Calculate solution

```
# calculate solution
solution = solve_ivp(lake_2box, [tini, tend], Cini, t_eval=t_space, method='RK45')
```

7) Plot or export results

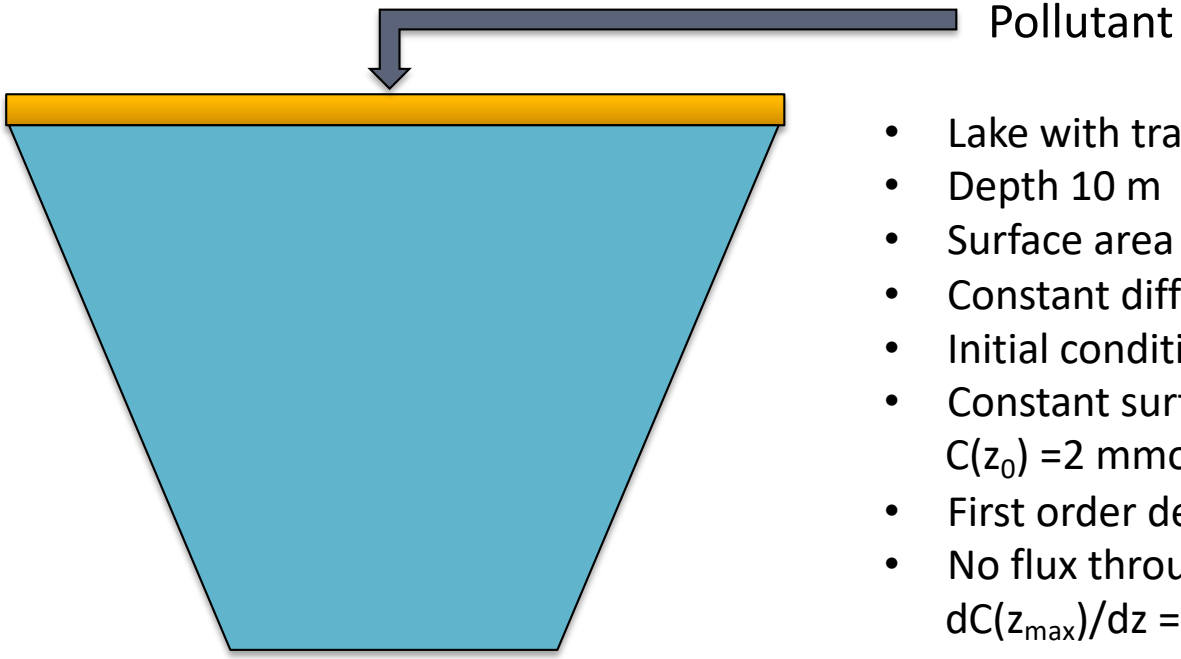
```
# plot results
plt.plot(solution.t, solution.y[0], label='epilimnion')
plt.plot(solution.t, solution.y[1], label='hypolimnion')
plt.legend()
plt.show()
```

How to define and solve the 2-box model in Python (2box_model.py)



Problems in space and time – Diffusion, mixing, advection

Box models assume homogeneously mixed compartments
But what if the spatiotemporal structure within a "box" is of interest?



- Lake with trapezoid (later: triangular) shape
- Depth 10 m
- Surface area 0.2 km^2, Bottom area 0.1 km^2
- Constant diffusivity K_z = 0.1 $m^2\ d^{-1}$
- Initial condition C_0= 0 at all depths
- Constant surface concentration: $C(z_0)$ =2 mmol L^{-1}
- First order decay k_r = 0.01 1 d^{-1}
- No flux through lower boundary: $dC(z_{max})/dz$ =0

Question:

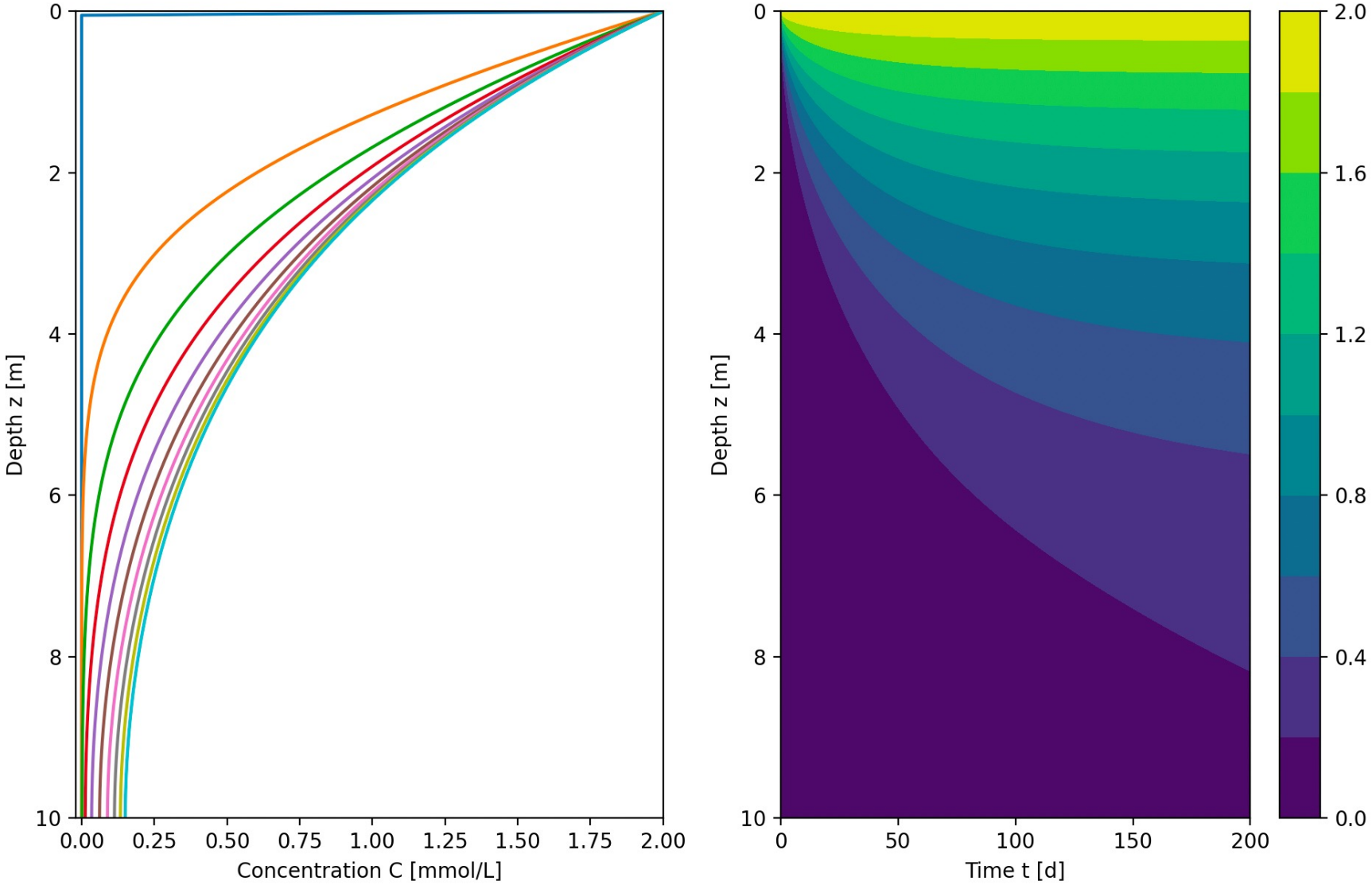
How will the concentration C develop as a function of time and depth?

Equation:

Diffusive transport — 1st order decay

$$A(z)\frac{\partial C}{\partial t} = \boxed{\frac{\partial}{\partial z}\left(K_z(z)A(z)\frac{\partial C}{\partial z}\right)} - \boxed{A(z)k_r C}$$

Problems in space and time – Results



Problems in space and time (simple_pde.py/ipynb)

Differential equation

```python
# PDE function
def simple_pde(z, C, t, dC_dz):
    # Define the vertical diffusivity
    Kz = 1e-1  # m2 / d

    # Define first order reaction rate
    kr = 1e-2  # [d - 1]

    # Define area function
    A = 200000 - 10000*z #trapezoid lake; area in m2

    # Define the PDE
    # A * dC/dt = Kz * A * d2C/dz2 - kr * A * C
    c = [A]
    f = [Kz * A * dC_dz]
    s = [-kr * A * C]

    return [c, f, s]
```

pde_solver (from pde import *)

```python
# Call pde_solver. It needs the following arguments:
# simple_pde: function containing the PDEs
# BC_left: BoundaryCondition containing conditions for z = 0
# BC_right: BoundaryCondition containing conditions for z = Zmax
# simple_IC: function containing initial condition for t=0 at all z
# z: z grid
# t: t grid
sol = pde_solver(simple_pde, BC_left, BC_right, simple_IC, z, t)
```

Boundary conditions

```python
# Boundary conditions:
# On the left, a dirichlet boundary condition C(t) = 2
# On the right, a neumann boundary condition dC/dz = 0
def neumann_function(t, u):
    return 0


# You need to have f=function for neumann conditions!
BC_left = BoundaryCondition(1, p=2)
BC_right = BoundaryCondition(2, f=neumann_function)
```

Initial conditions

```python
# Initial conditions
def simple_IC(x):
    # Initial conditions for t = 0
    # Here we set C0 = 0
    return [0]
```

Time and space discretization

```python
Zmax = 10  # Lake depth [m]
tmax = 200  # maximum simulation time [d]

M = 2001  # number of timesteps
N = 201  # number of spacial nodes

t = np.linspace(0, tmax, M)  # time discretization
z = np.linspace(0, Zmax, N)  # space discretization
```

Plot the output

```python
# plot showing time
plt.figure(1)
ax = plt.subplot(1, 2, 1)
plt.ylim(0, 10)
plt.xlim(-0.02, 2.0)
ax.invert_yaxis()
plt.xlabel('Concentration C [mmol/L]')
plt.ylabel('Depth z [m]')
for i in range(0, int(len(t) / 200)):
    plt.plot(sol[0, :, i * 200], z)

# contour plot
ax = plt.subplot(1, 2, 2)
ax.invert_yaxis()
plt.xlabel('Time t [d]')
plt.ylabel('Depth z [m]')
plt.contourf(t, z, sol[0], 10)
plt.colorbar()
plt.show()
```

Problems in space and time – Initial and boundary conditions

Initial condition:

$$C(t_0) = 0$$

```
# Initial conditions
def simple_IC(x):
    # Initial conditions for t = 0
    # Here we set C0 = 0
    return [0]
```

Boundary conditions:

$$C(z_0) = 2$$

$$\frac{\partial C}{\partial z}(z_{max}) = 0$$

```
# Boundary conditions:
# On the left, a dirichlet boundary condition C(t) = 2
# On the right, a neumann boundary condition dC/dz = 0
def neumann_function(t, u):
    return 0


# You need to have f=function for neumann conditions!
BC_left = BoundaryCondition(1, p=2)
BC_right = BoundaryCondition(2, f=neumann_function)
```

Boundary conditions:

Dirichlet (type 1): $u(t) = p + q(t,u) * t$
Known value at boundary

Neumann (type 2): $\frac{\partial u}{\partial x} = f(t,u)$
Known flux at boundary

Problems in space and time – Differential equation

$$\underbrace{A(z)}_{c}\frac{\partial C}{\partial t} = \underbrace{\frac{\partial}{\partial z}\left(\overbrace{K_z(z)A(z)\frac{\partial C}{\partial z}}^{f}\right)}_{\text{Diffusive transport}} - \underbrace{\overbrace{A(z)k_r C}^{s}}_{1^{st}\text{ order decay}}$$

```python
# PDE function
def simple_pde(z, C, t, dC_dz):
    # Define the vertical diffusivity
    Kz = 1e-1  # m2 / d

    # Define first order reaction rate
    kr = 1e-2  # [d - 1]

    # Define area function
    A = 200000 - 10000*z #trapezoid lake; area in m2

    # Define the PDE
    # A * dC/dt = Kz * A * d2C/dz2 - kr * A * C
    c = [A]
    f = [Kz * A * dC_dz]
    s = [-kr * A * C]

    return [c, f, s]
```

Careful:
c, f, and s must be lists or arrays even when only solving a single PDE!

Problems in time and space – Grid and solver

Defines the grid

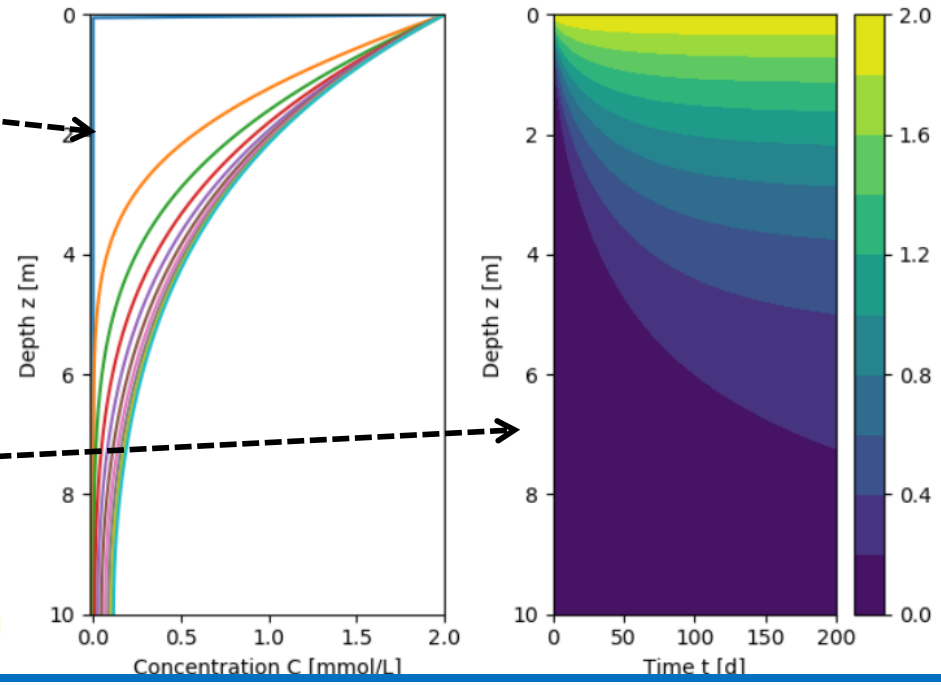
```
Zmax = 10   # Lake depth [m]
tmax = 200   # maximum simulation time [d]

M = 2001   # number of timesteps
N = 201   # number of spacial nodes

t = np.linspace(0, tmax, M)   # time discretization
z = np.linspace(0, Zmax, N)   # space discretization
```

Solves the equation

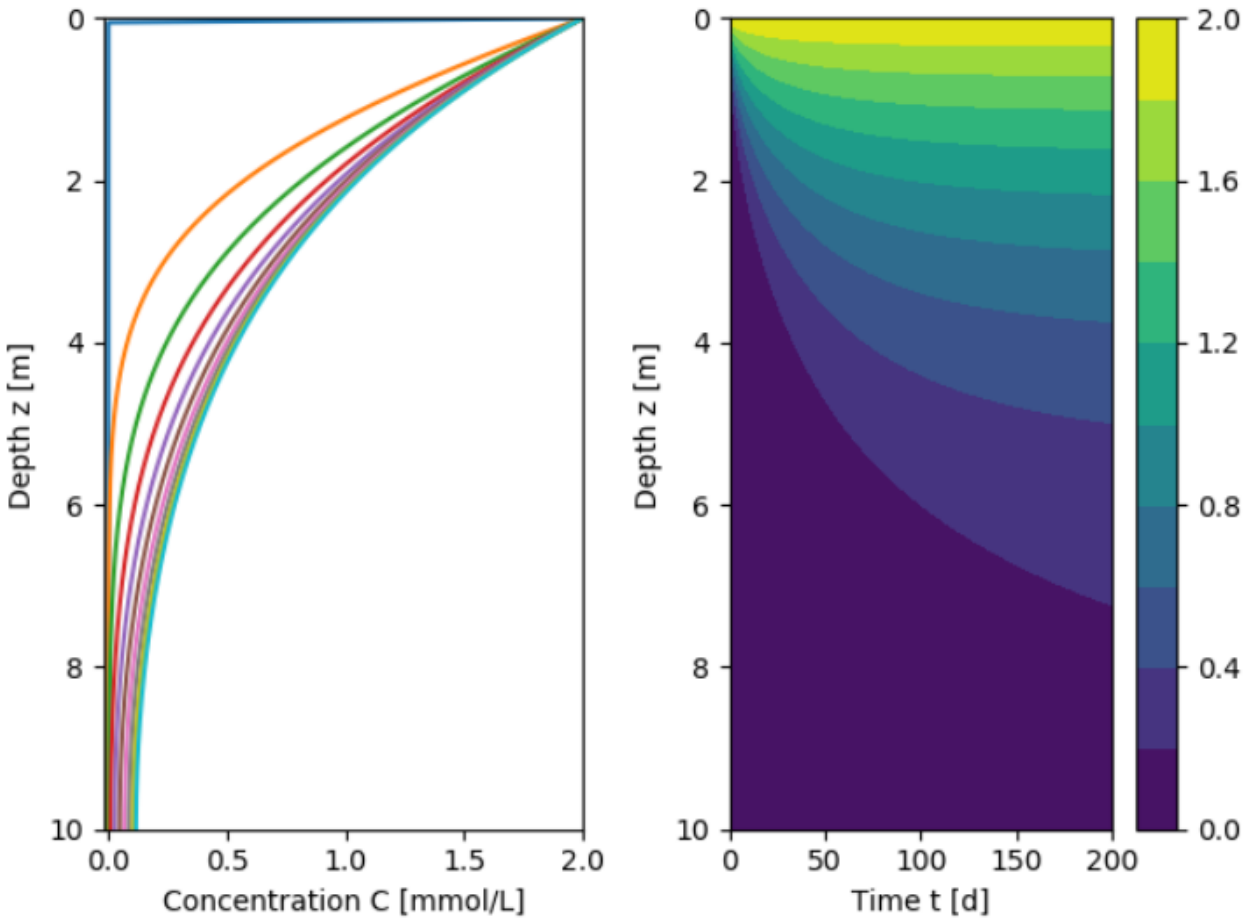
```
# Call pde_solver. It needs the following arguments:
# simple_pde: function containing the PDEs
# BC_left: BoundaryCondition containing conditions for z = 0
# BC_right: BoundaryCondition containing conditions for z = Zmax
# simple_IC: function containing initial condition for t=0 at all z
# z: z grid
# t: t grid
sol = pde_solver(simple_pde, BC_left, BC_right, simple_IC, z, t)
```

Problems in time and space – Plotting the results

```
# plot showing time
plt.figure(1)
ax = plt.subplot(1, 2, 1)
plt.ylim(0, 10)
plt.xlim(-0.02, 2.0)
ax.invert_yaxis()
plt.xlabel('Concentration C [mmol/L]')
plt.ylabel('Depth z [m]')
for i in range(0, int(len(t) / 200)):
    plt.plot(sol[0, :, i * 200], z)

# contour plot
ax = plt.subplot(1, 2, 2)
ax.invert_yaxis()
plt.xlabel('Time t [d]')
plt.ylabel('Depth z [m]')
plt.contourf(t, z, sol[0], 10)
plt.colorbar()
plt.show()
```



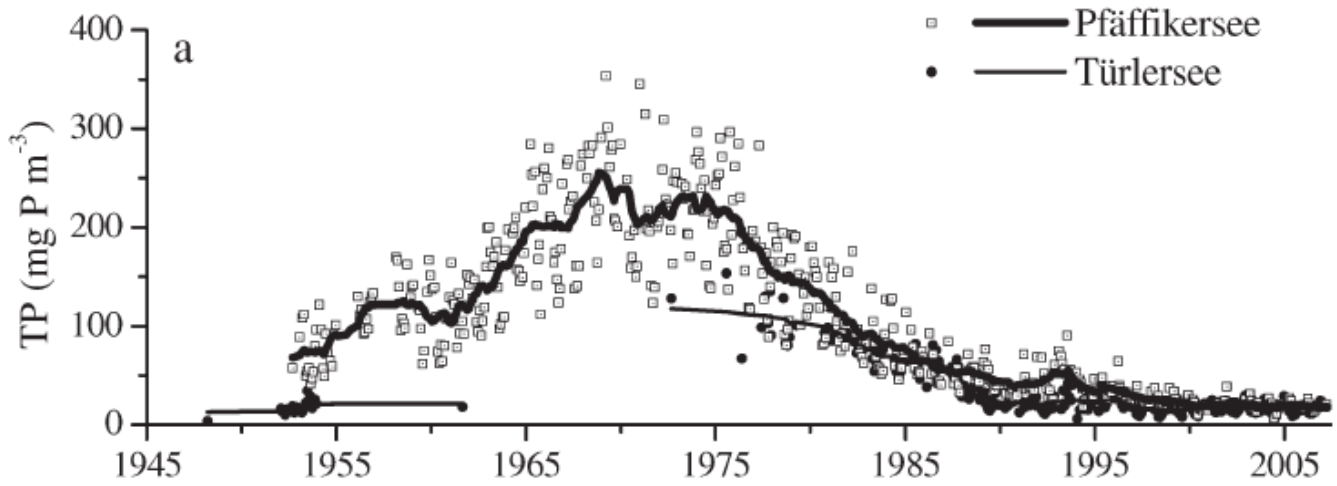
Problems in time and space – Exercise 4



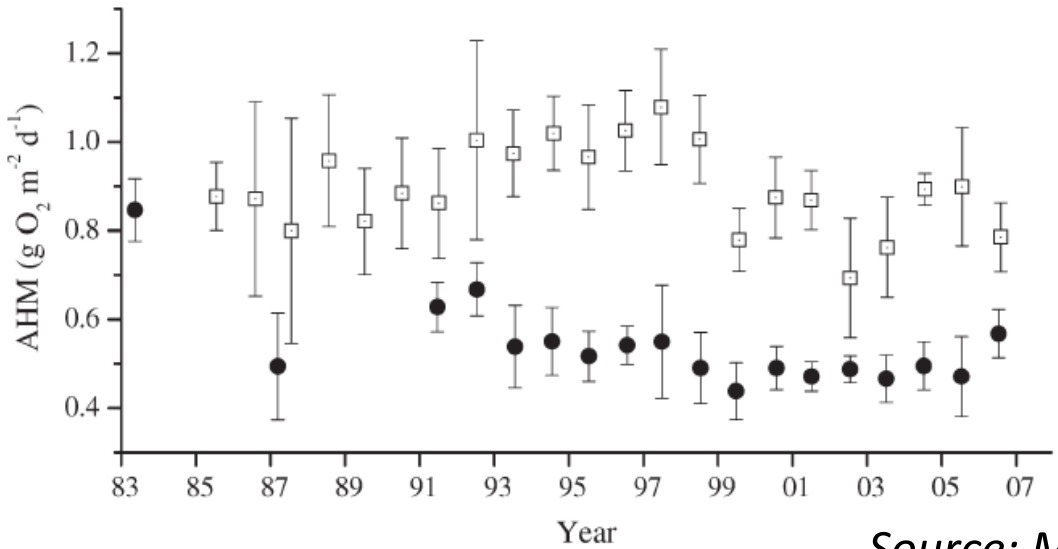
Exercise 4: Make your lake triangular (solution: simple_pde_corr.py; simple_fipy_corr.py)

Homework: Exercise 1 (submit code & answers on March 13 on Moodle)

Large success in reducing phosphorus inputs to lakes…



… but not much change in hypolimnetic oxygen consumption.

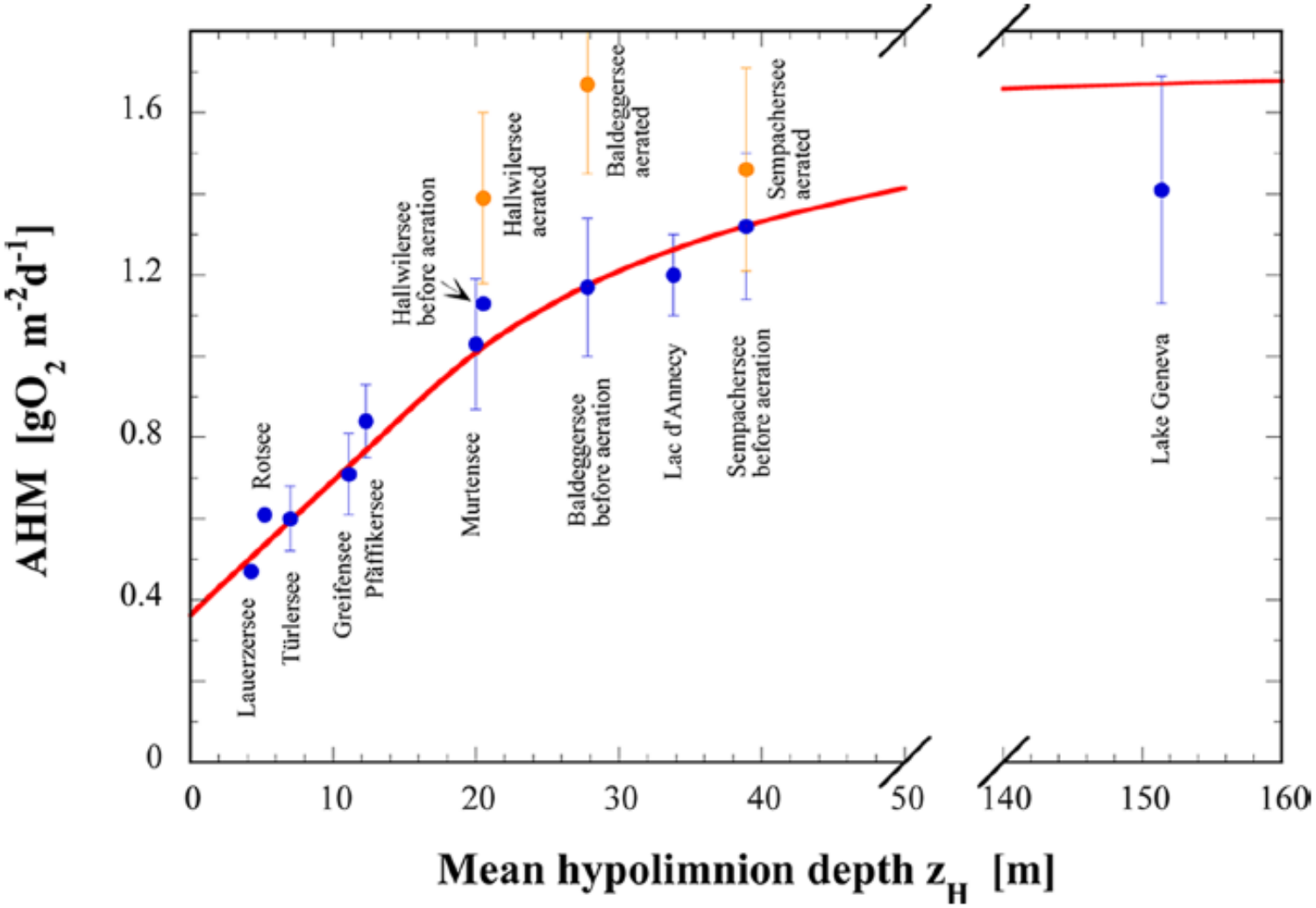


AHM (Areal Hypolimnetic Mineralization)
Sum of oxygen consumption and oxygen equivalents of accumulation of reduced substances (methane, ammonium, …)

Source: Matzinger et al. (2010) L&O 55, 2073-2084

Homework: Exercise 1

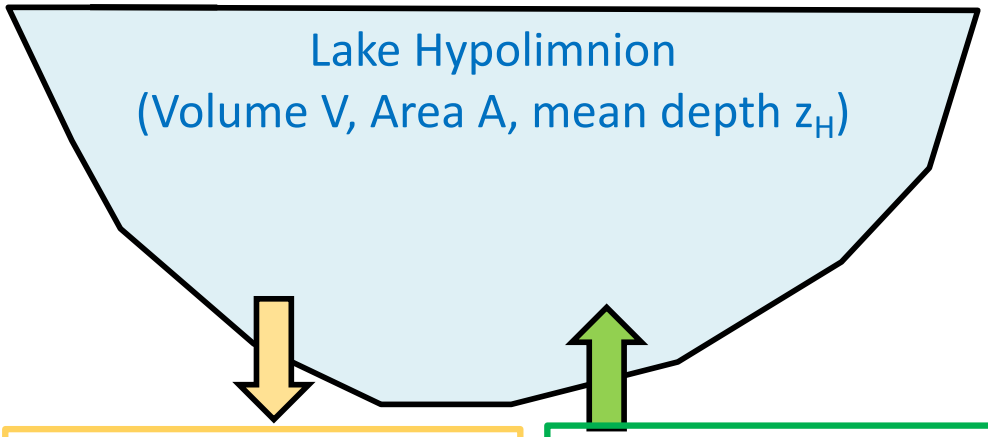
AHM (Areal Hypolimnetic Mineralization) mainly depends on lake depth, as long as lakes are not oligotrophic.



Source: Müller et al. (2012) ES&T 46, 9964-9971

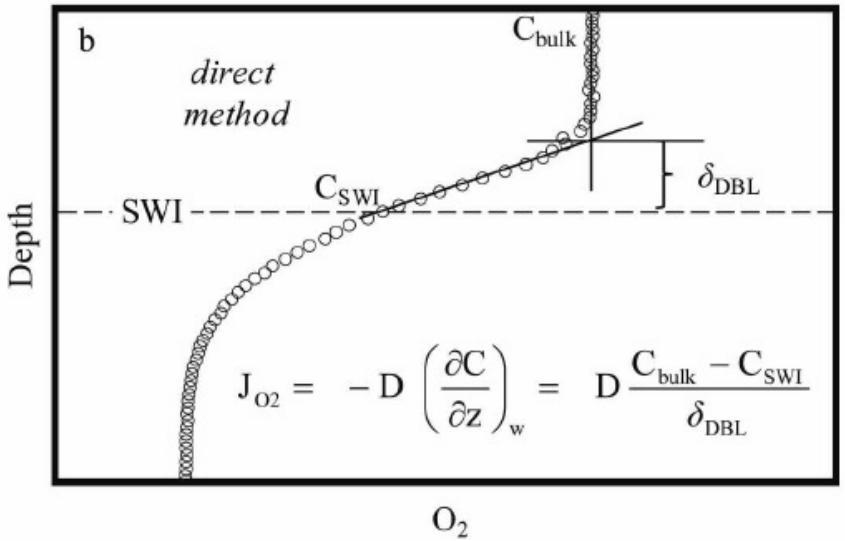
Homework Exercise 1

Simple 1-box model of the hypolimnion to explain this.



F_{O2}: Flux of oxygen towards the sediment surface (used for aerobic mineralization), depends on oxygen concentration.

F_{red}: Flux of reduced substances from the sediment, expressed in oxygen equivalents, and assumed to be constant in time.



Diffusive flux of oxygen through the sediment-water interface; here we assume that the O_2 concentration at the sediment surface is zero.
(Bryant et al. (2012) L&O Meth. 8, 610-627)

1-box model for oxygen concentration C

$$V\frac{dC(t)}{dt} = -AF_{red} - \frac{A * D_{O_2}}{\delta}C(t) \quad \text{and with} \quad z_H = \frac{V}{A}$$

$$\boxed{\frac{dC(t)}{dt} = -\frac{1}{z_H}F_{red} - \frac{D_{O_2}}{\delta z_H}C(t)}$$